so-04-risorse.pdf

GESTIONE RISORSE E DEADLOCK

Risorse
Un sistema di elaborazione è composto da un insieme di risorse da assegnare ai processi presenti
I processi competono nell'accesso alle risorse
Esempi di risorse
○ *memoria* ○ *stampanti* ○ *processore* ○ *dischi* ○ *interfaccia di rete* ○ *descrittori di processo*

Classi di risorse

Le risorse possono essere suddivise in classi → le risorse appartenenti alla stessa classe sono equivalenti
esempi: byte della memoria, stampanti dello stesso tipo, etc.
Definizioni:

- le risorse di una classe vengono dette istanze della classe
- il numero di risorse in una classe viene detto molteplicità del tipo di risorsa

Un processo non può richiedere una specifica risorsa, ma solo UNA RISORSA DI UNA SPECIFICA CLASSE
- una richiesta per una classe di risorse può essere soddisfatta da qualsiasi istanza di quel tipo

Assegnazione delle risorse

Risorse ad assegnazione statica ⟹ Avviene al momento della creazione del processo e rimane valida fino alla terminazione
esempi:
descrittori di processi, aree di memoria (in alcuni casi)
Risorse ad assegnazione dinamica ⟹ I processi richiedono le risorse durante la loro esistenza
→ le utilizzano una volta ottenute e le rilasciano quando non più necessarie (eventualmente alla terminazione del processo
esempi:
periferiche di I/O, aree di memoria (in alcuni casi)

Tipi di richieste

Richiesta singola: si riferisce a una singola risorsa di una classe definita
⇒è il caso normale
Richiesta multipla: si riferisce a una o più classi, e per ogni classe, ad una o più risorse
⇒deve essere soddisfatta integralmente

Richiesta bloccante: il processo richiedente si sospende se non ottiene immediatamente l'assegnazione ⟹ la richiesta rimane pendente e viene riconsiderata dalla funzione di gestione ad ogni rilascio
Richiesta non bloccante: la mancata assegnazione viene notificata al processo richiedente, senza provocare la sospensione

Risorse seriali (o con accesso mutuamente esclusivo): una singola risorsa non può essere assegnata a più processi contemporaneamente
esempi:
■ *i processori* ■ *le sezioni critiche* ■ *le stampanti*

Risorse non seriali
esempio:
■ *file di sola lettura*

Risorse pre-rilasciabili ("preemptable") ⟹ Una risorsa si dice **PRE-RILASCIABILE** se la funzione di gestione può sottrarla ad un processo prima che questo l'abbia effettivamente rilasciata
Meccanismo di gestione: il processo che subisce il pre-rilascio deve sospendersi ⟶ la risorsa pre-rilasciata sarà successivamente restituita al processo

UNA RISORSA E' PRE-RILASCIABILE: se il suo stato non si modifica durante l'utilizzo → oppure il suo stato può essere facilmente salvato e ripristinato
Esempi:
○ *processore* ○ *blocchi o partizioni di memoria* (nel caso di assegnazione dinamica)

Risorse NON PRE-RILASCIABILI
Definizione: la funzione di gestione non può sottrarle al processo al quale sono assegnate
→sono **NON PRE-RILASCIABILI** le risorse il cui stato non può essere salvato e ripristinato
Esempi:
○ *stampanti* ○ *classi di sezioni critiche* ○ *partizioni di memoria* (nel caso di gestione statica)

Deadlock

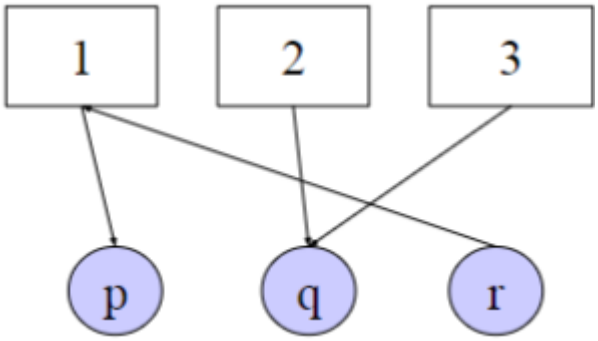
I deadlock impediscono ai processi di terminare correttamente, e le risorse bloccate in deadlock non possono essere utilizzati da altri processi.

Condizioni per avere un deadlock

Mutua esclusione : le risorse coinvolte devono essere seriali
Assenza di pre-rilascio : le risorse coinvolte non possono essere pre-rilasciate, ovvero devono essere rilasciate volontariamente dai processi che le controllano
Richieste bloccanti (detta anche "hold and wait") le richieste devono essere bloccanti, e un processo che ha già ottenuto risorse può chiederne ancora
Attesa circolare : esiste una sequenza di processi P0 ,P1 , ..., Pn , tali per cui P 0 attende una risorsa controllata da P1 , P1 attende una risorsa controllata da P2 , ..., e Pn attende una risorsa controllata da P0
L'insieme di queste condizioni è necessario e sufficiente
→devono valere tutte contemporaneamente affinché un deadlock si presenti nel sistema

Grafo di Holt

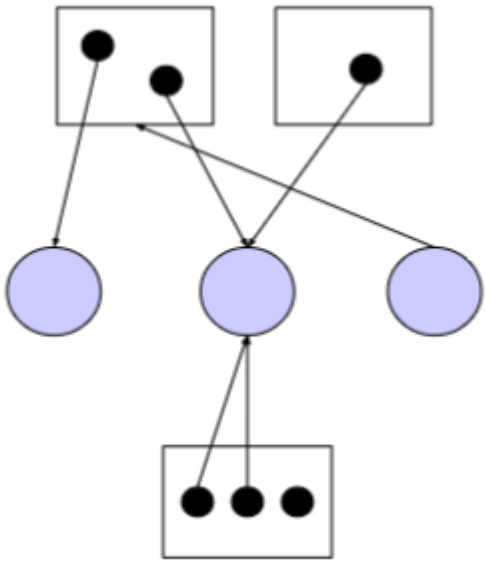
E' UN GRAFO DIRETTO ⟹ gli archi hanno una direzione
E' UN GRAFO BIPARTITO ⟹ i nodi sono suddivisi in due sottoinsiemi e non esistono archi che collegano nodi dello stesso sottoinsieme ⇒ i sottoinsiemi sono RISORSE e PROCESSI
gli archi *risorsa → processo* indicano che la risorsa è assegnata al processo
gli archi *processo → risorsa* indicano che il processo ha richiesto la risorsa
risorsa → processo la risorsa è in uso
processo → risorsa il processo aspetta la risorsa



cerchi processi , quadrati risorse

Nel caso di classi contenenti più istanze di una risorsa **L'INSIEME DELLE RISORSE E' PARTIZIONATO IN CLASSI** e gli archi di richiesta sono diretti alla classe e non alla singola risorsa
Rappresentazione i processi sono rappresentati da cerchi, le classi sono rappresentati come contenitori rettangolari ⇒ le risorse come punti all'interno delle classi

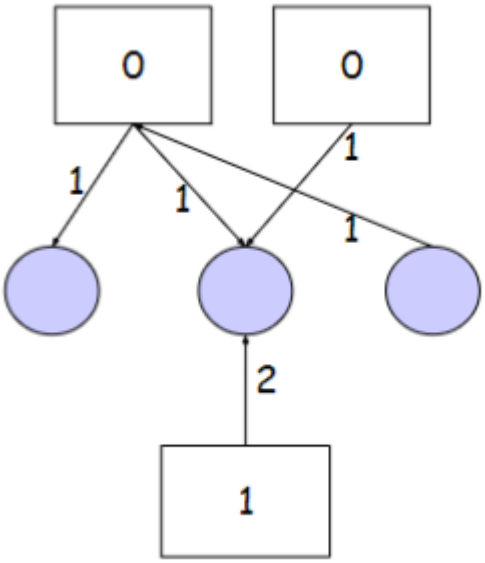
Nota: non si rappresentano grafi di Holt con archi relativi a richieste che possono essere soddisfatte , se esiste almeno un'istanza libera della risorsa richiesta, la risorsa viene assegnata



notazione alternativa

alcuni autori preferiscono la notazione numerica

sugli archi: la molteplicità della richiesta (archi processo → classe)
la molteplicità dell'assegnazione (archi classe → processo)
all'interno delle classi il numero di risorse non ancora assegnate



Metodi di gestione dei deadlock

Deadlock detection and recovery permettere al sistema di entrare in stati di deadlock; utilizzare un algoritmo per rilevare questo stato ed eventualmente eseguire un'azione di recovery

Deadlock prevention / **avoidance** impedire al sistema di entrare in uno stato di deadlock
Ostrich algorithm ignorare il problema del tutto!

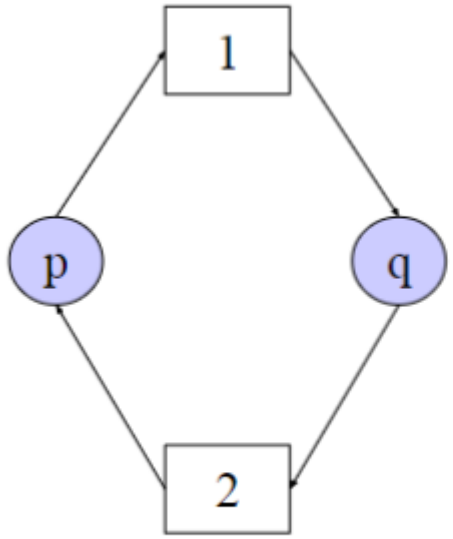
Deadlock detection

Descrizione mantenere aggiornato il grafo di Holt, registrando su di esso tutte le assegnazioni e le richieste di risorse
UTILIZZIAMO IL GRAFO DI HOLT AL FINE DI RICONOSCERE GLI STATI DI DEADLOCK
Problema: come riconoscere uno stato di deadlock?

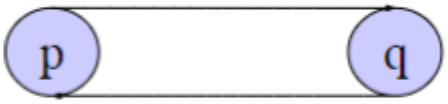
Caso 1 - Una sola risorsa per classe

Teorema se le risorse sono ad *ACCESSO MUTUALMENTE ESCLUSIVO, SERIALI E NON PRE-RILASCIABILI*
→ Lo stato **E' DI DEADLOCK** se e solo se il grafo di Holt contiene un ciclo.

Dimostrazione
si utilizza una variante del grafo di Holt, detto grafo Wait-For
→si ottiene un grafo *wait-for* **ELIMINANDO I NODI DI TIPO RISORSA** e collassando gli archi appropriati
⟶ il grafo di Holt contiene un ciclo se e solo se il grafo Wait-for contiene un ciclo
⟶ se il grafo Wait-for contiene un ciclo, ***ABBIAMO ATTESA CIRCOLARE***



Grafo di Holt



Grafo Wait-For

Esercizio 1
Considera un sistema con 4 processi (P1, P2, P3, P4) e 3 risorse (R1, R2, R3) - Stato iniziale delle risorse
R1: allocata a P1
R2: allocata a P2

R3: libera

Sequenza di Richieste

P3 richiede R1

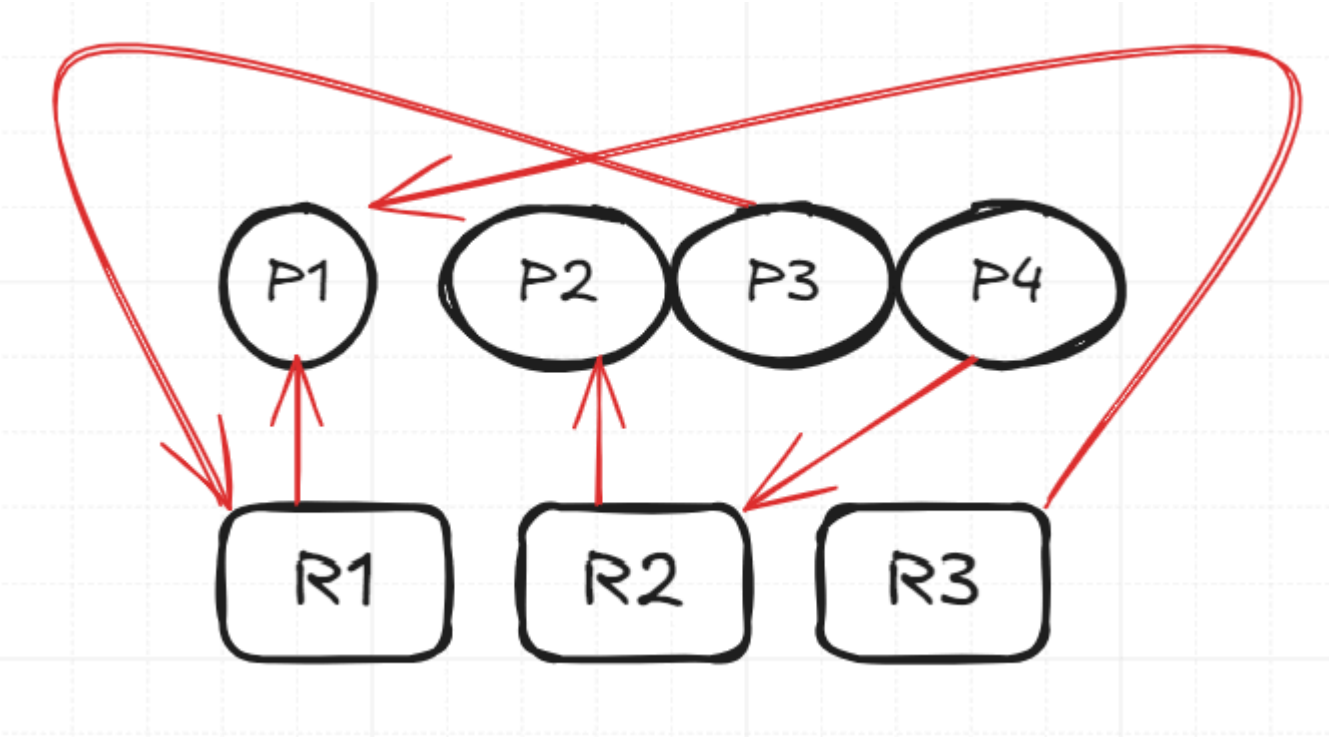
P4 richiede R2

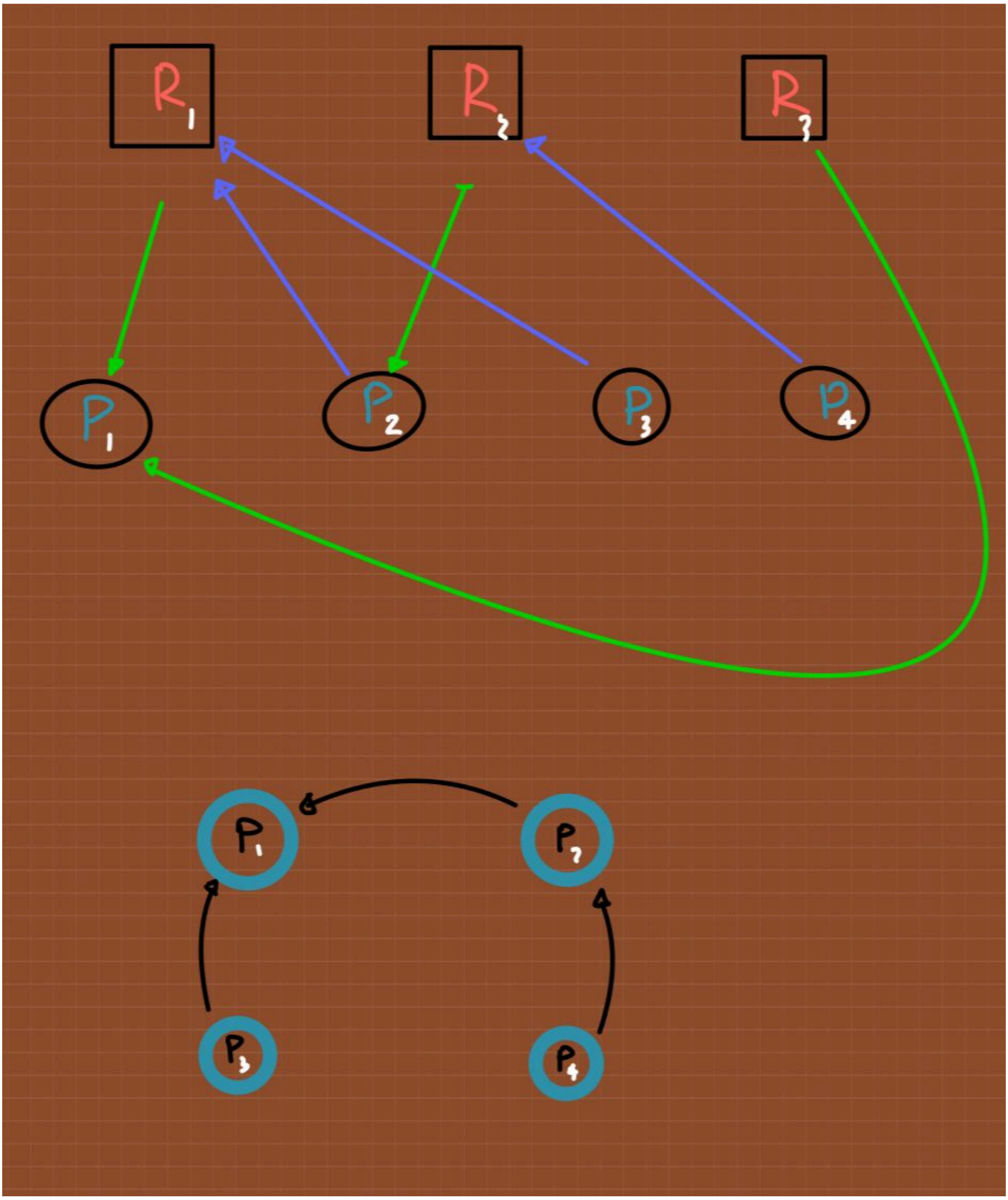
P1 richiede R3

P2 richiede R1

Richieste per lo studente Costruire il grafo wait-for iniziale

→Verificare se esiste una situazione di deadlock

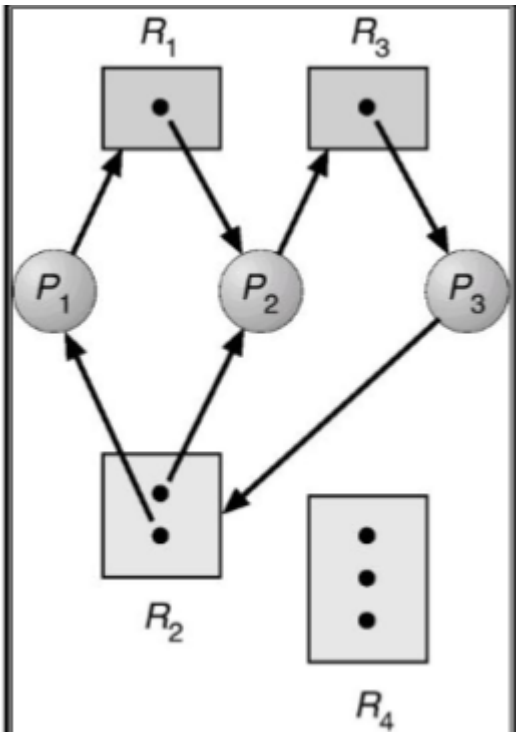




NO CICLO IN GRAFO FOR-WAIT QUINDI NO CICLO IN GRAFO DI HOLT QUINDI NO SITUAZIONE DI DEADLOCK

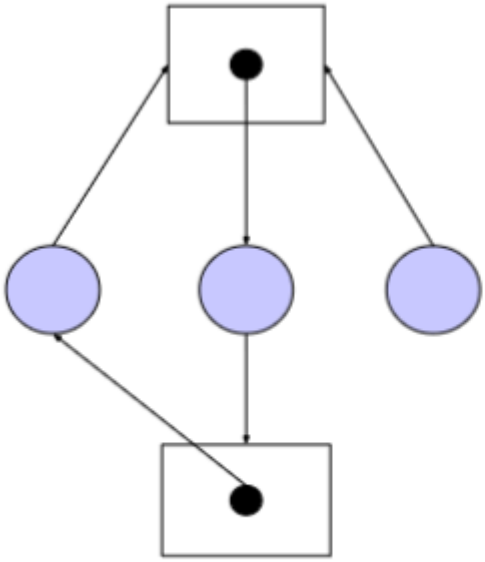
Esercizio 2

C'è deadlock? SI

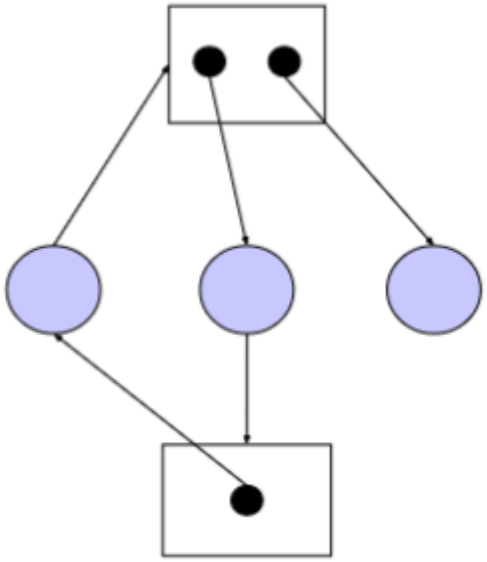


Caso 2 - Più risorse per classe

La presenza di un ciclo nel caso di Holt non è condizione sufficiente per avere deadlock



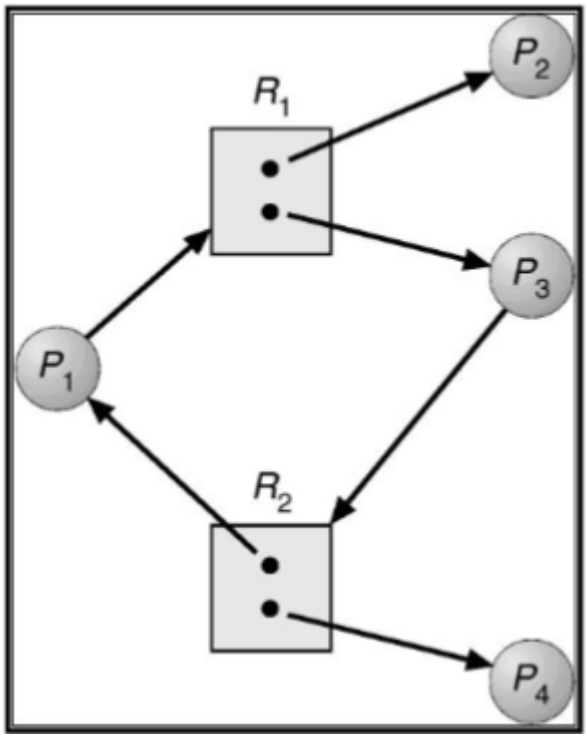
Deadlock



No Deadlock

nel momento in cui abbiamo piu istanze ⟶ per avere una deadlock si è necessario un cicli, ma non è piu sufficiente
quindi se ho un numero di istanze(o istanze) = numero di processi no deadlock
(nel caso a destra)quando la risorsa3 occupata dal processo verrà rilasciata , questa verrà allocata (destinata) per un altro processo, sbloccando il ciclo

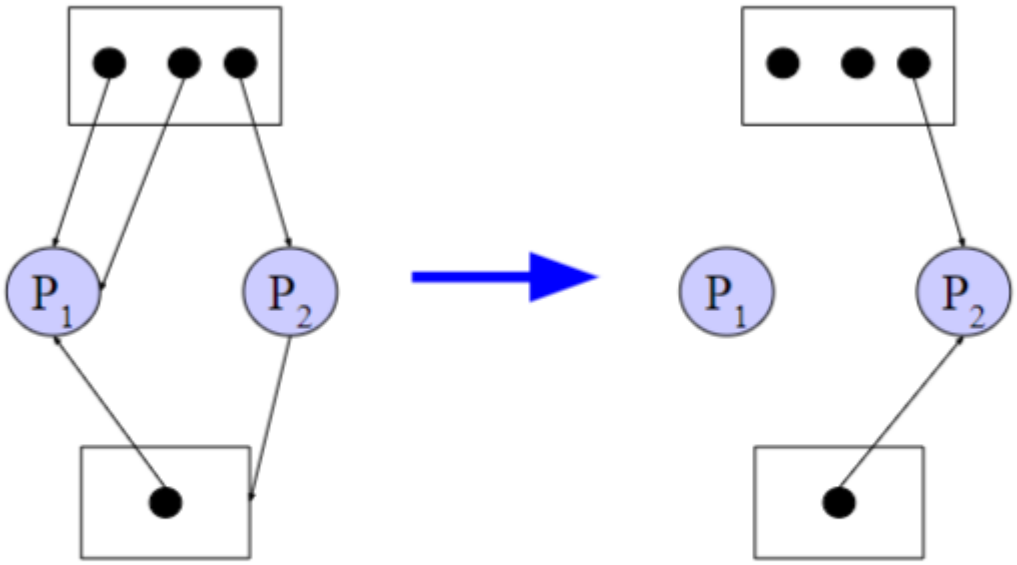
Esercizio 3 C'è deadlock?



no , perché ci sono processi che rilasceranno le risorse , disponibili poi quindi per i processi che sono coinvolti in un ciclo
P4 termina e rilascia la risorsa , disponibile per P3 , P2 termina e lascia la risorsa disponibile per P1, dopodiché P1 e P3 possono terminare

Riducibilità di un grafo di Holt
Un grafo di Holt si dice RIDUCIBILE *se esiste almeno un NODO PROCESSO con solo archi entranti*
Riduzione ⟹ consiste nell'eliminare tutti gli archi di tale nodo e riassegnare le risorse ad altri processi
Qual è la logica?
eventualmente, un nodo che utilizza una risorsa prima o poi la rilascerà; a quel punto, la risorsa può essere riassegnata

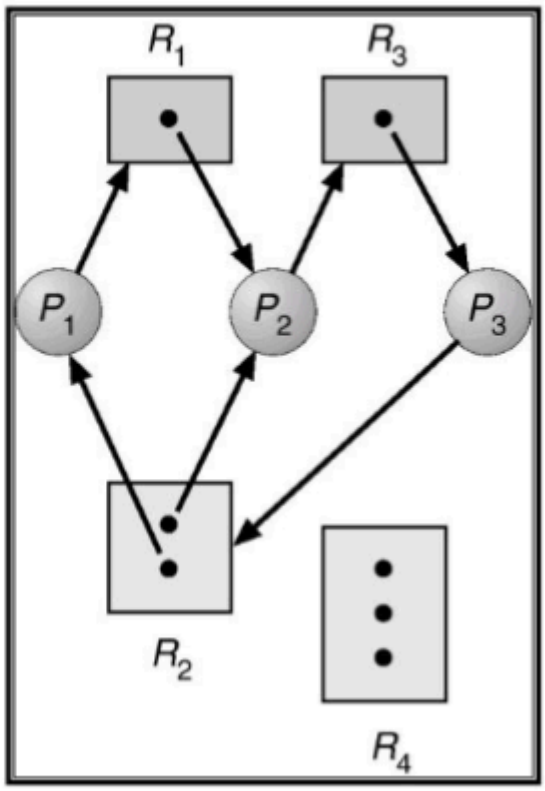
Esempio di riduzione



Riduzione per P_1

ESERCIZIO

C'è deadlock?



Dal grafo:

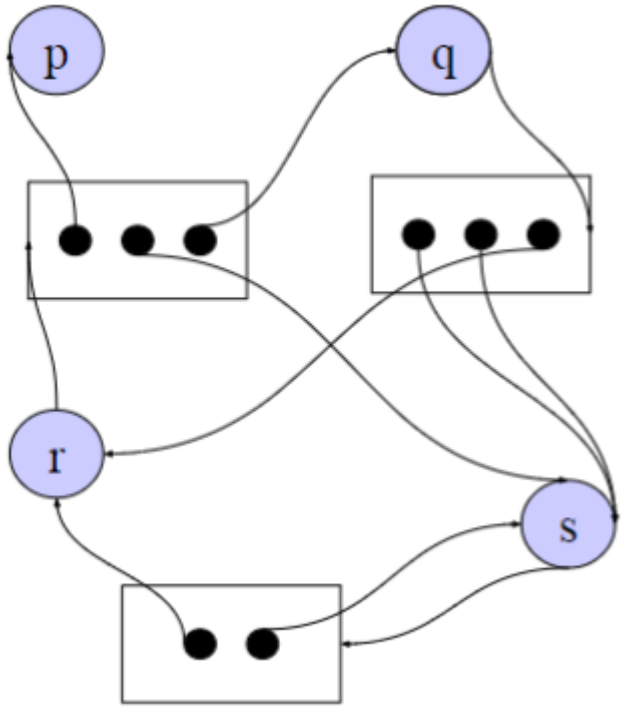
- P1 attende risorse da R1 e R2.
- P2 attende risorse da R2 e R3.

- P3 attende risorse da R3.
  Le frecce mostrano che esiste un ciclo:
- P1 → R2 → P2 → R3 → P3 → R1 → P1.
  QUINDI SI... c'é deadlock

Deadlock detection con grafo di Holt

Teorema se le risorse sono ad accesso mutualmente esclusivo, seriali e non pre-rilasciabili
→ LO STATO NON E' DI DEADLOCK se e solo se il grafo di Holt è COMPLETAMENTE RIDUCIBILE se solo se esiste una sequenza di passi di riduzione che elimina tutti gli archi del grafo

Esercizio 2
c'é deadlock?



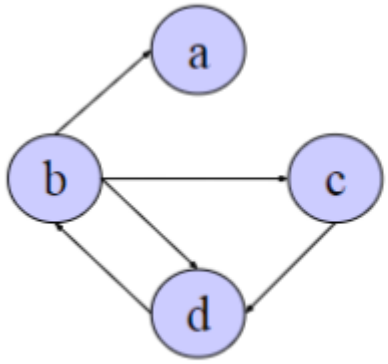
- Il processo **p** può terminare perché tutte le risorse che utilizza sono disponibili.
- Una volta che **p** termina, rilascia le risorse che possono essere usate da **r**.
- Dopo che **r** termina, rilascia altre risorse necessarie per **s** e **q**, consentendo anche a loro di completare.

1. Riduzione di p Assegnamento risorsa a r
2. Riduzione di r Assegnamento risorse a q,s
3. Riduzione di q
4. Riduzione di s

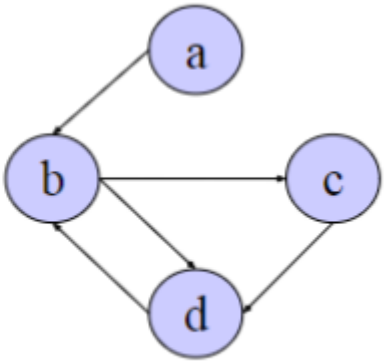
Quindi, non c'è alcun ciclo permanente di attesa reciproca, e i processi possono progredire. **Non c'è deadlock.**

Deadlock detection - Knot

Dato un nodo n, **L'INSIEME DEI NODI RAGGIUNGIBILI DA n VIENE DETTO** *INSIEME DI RAGGIUNGIBILITA' DI n* (scritto R(n))
un knot del grafo G è il massimo sottoinsieme (non banale) di nodi M tale che per ogni n in M, R(n)=M
in altre parole: partendo da un qualunque nodo di M, si possono raggiungere tutti i nodi di M e nessun nodo all'infuori di esso.



{ b,c,d } non è un knot



{ b,c,d } è un knot

Teorema dato un grafo di Holt con una sola richiesta sospesa per processo
se le risorse sono ad accesso mutualmente esclusivo, seriali e non prerilasciabili,
→allora il grafo rappresenta uno stato di deadlock se e solo se esiste un knot

Deadlock recovery

Dopo aver rilevato un deadlock
bisogna risolvere la situazione
→ La soluzione può essere:

- **MANUALE**
  ■ l'operatore viene informato e eseguirà alcune azioni che permettano al sistema di proseguire
- **AUTOMATICA**
  ■ il sistema operativo è dotato di meccanismi che permettono di risolvere in modo automatico la situazione, in base ad alcune politiche

Meccanismi per il deadlock recovery

Terminazione totale: tutti i processi coinvolti vengono terminati
Terminazione parziale: viene eliminato un processo alla volta, fino a quando il deadlock non scompare
Preemption: una risorsa (o più di una, se necessario) viene sottratta ad uno dei processi

coinvolti nel deadlock
Checkpoint/rollback lo stato dei processi viene periodicamente salvato su disco (checkpoint) : in caso di deadlock, si ripristina (rollback) uno o più processi ad uno stato precedente, fino a quando il deadlock non scompare

Considerazioni
Terminare processi può essere costoso, questi processi possono essere stati eseguiti per molto tempo ⟹ se terminati, dovranno ripartire da capo.
Terminare processi può lasciare le risorse in uno stato inconsistente se un processo viene terminato nel mezzo di una sezione critica
Fare preemption ; non sempre è possibile, può richiedere interventi manuali
esempio:
preemption di una stampante

DEADLOCK PREVENTION / AVOIDANCE
Prevention
per evitare il deadlock si elimina una delle quattro condizioni del deadlock
→ il deadlock viene eliminato STRUTTURALMENTE
Avoidance
prima di assegnare una risorsa ad un processo, si controlla se l'operazione può portare al pericolo di deadlock
→ in quest'ultimo caso, L'OPERAZIONE VIENE RITARDATA

Deadlock prevention

ATTACCARE LA CONDIZIONE di "MUTUA ESCLUSIONE"
permettere la condivisione di risorse →e.g. spool di stampa, tutti i processi "pensano" di usare contemporaneamente la stampante

PROBLEMI DELLO SPOOLING in generale, lo spooling non sempre è applicabile → si sposta il problema verso altre risorse
■ il disco nel caso di spooling di stampa
■ cosa succede se due processi che vogliono stampare due documenti esauriscono lo spazio su disco?

ATTACCARE LA CONDIZIONE di "RICHIESTA BLOCCANTE"
ALLOCAZIONE TOTALE
■ è possibile richiedere che un processo richieda tutte le risorse all'inizio della computazione
Problemi
■ non sempre l'insieme di richieste è noto fin dall'inizio
■ si riduce il parallelismo

Attaccare la condizione di "Assenza di pre-rilascio"
non sempre è possibile → può richiedere interventi manuali

Attaccare la condizione di "Attesa Circolare"
Allocazione gerarchica ⇒ alle classi di risorse vengono associati valori di priorità →ogni processo in ogni istante può allocare solamente risorse di priorità superiore a quelle che già possiede
→se un processo vuole allocare una risorsa a priorità inferiore, deve prima rilasciare tutte le risorse con priorità uguale o superiore a quella desiderata

ALLOCAZIONE GERARCHICA E ALLOCAZIONE TOTALE: *problemi*
prevengono il verificarsi di deadlock, ma sono altamente inefficienti
Nell'allocazione gerarchica:
l'indisponibilità di una risorsa ad alta priorità ritarda processi che già detengono risorse ad alta priorità
Nell'allocazione totale:
anche se un processo ha necessità di risorse per poco tempo deve allocarla per tutta la propria esistenza

Deadlock Avoidance
L'algoritmo del banchiere ⟶ un algoritmo per evitare lo stallo sviluppato da Dijkstra (1965)
Il nome deriva dal metodo utilizzato da un *ipotetico banchiere* di provincia che *gestisce un gruppo di clienti a cui ha concesso del credito*; **non tutti i clienti avranno bisogno dello stesso credito simultaneamente**
Descrizione
un banchiere desidera condividere un capitale (fisso) con un numero (prefissato) di clienti

- per Dijkstra l'"unità di misura" erano fiorini olandesi
- ogni cliente specifica in anticipo la sua necessità massima di denaro
  - che ovviamente non deve superare il capitale del banchiere
- i clienti fanno due tipi di transazioni, **richieste di prestito, restituzioni**
- il denaro prestato ad ogni cliente non può mai eccedere la necessità massima specificata a priori
- ogni cliente può fare richieste multiple, fino al massimo importo specificato
- una volta che le richieste sono state accolte e il denaro è stato ottenuto deve garantire la restituzione in un tempo finito

N:	numero dei clienti
IC:	capitale iniziale
c_i:	limite di credito del cliente i ($c_i < IC$)
p_i:	denaro prestato al cliente i ($p_i \leq c_i$)
$n_i = c_i - p_i$	credito residuo del cliente i
$SC = IC - \sum_{i=1..N} p_i$	saldo di cassa

Stato SAFE (Informale)

Le richieste che ogni cliente i può fare possono essere soddisfatte dalle risorse attualmente disponibili più tutte le risorse detenute dai processi j soddisfatti precedentemente (j<i)

Definizione: *Stato SAFE*

- sia s una permutazione dei valori 1...N
  - esempio, con N=4: s = 1 , 3 , 4 , 2
  - indichiamo con s(i) l'i-esima posizione della sequenza
- si calcoli il vettore avail come segue
  - avail[1] = *SC*
  - avail[j+1] = avail[j] + $p_{s(j)}$, con j=1...N-1
- uno stato del sistema si dice safe se vale la seguente condizione:
  - $n_{s(j)} \leq$ avail[j], con j=1...N

→ Finché 1 non termina, ho a disposizione solo il saldo in cassa (SC)
→Quello che ho a disposizione per soddisfare la richiesta del cliente j+1 è quello che avevo prima del cliente j sommato a quello che j mi restituirà dopo la sua richiesta
→sono tranquillo, se date le massime richieste possibili, riesco a trovare un ordinamento dei clienti per cui, se gestiti in sequenza, ho sempre disponibilità

Lo stato UNSAFE è condizione necessaria ma non sufficiente per avere deadlock i.e., un sistema in uno stato UNSAFE può evolvere senza procurare alcun deadlock

Capitale Iniziale	IC	150

Saldo di cassa	SC	150

Cliente	MAX	Prestito Attuale	Credito residuo
i	c_i	p_i	n_i
1	100	0	100
2	20	0	20
3	30	0	30
4	50	0	50
5	70	0	70

Regola pratica (per il banchiere a singola valuta) lo stato SAFE può essere verificato usando la sequenza che ordina in modo crescente i valori di n i (credito residuo di i) sono sicuro che concedo ad ogni passo il minimo possibile per poi recuperarlo e aumentare la disponibilità di prestito per il prossimo cliente

Cliente	MAX	Prestito Attuale	Credito residuo
i	c_i	p_i	n_i
1	100	70	30
2	20	10	10
3	30	30	0
4	50	10	40
5	70	30	40

Immaginiamo di essere arrivati a questo stato di prestiti. Per valutare se siamo in uno stato safe: dobbiamo considerare il caso pessimo delle richieste, i.e., massime richieste possibili (i.e., residui n i) Ordiniamo i processi sulla base dei loro n

---

note +
guardiamo 3→ ha credito residuo 0 , quindi sarà il primo a restitire⟶ poi possiamo

soddisfare 2 , che al termine restituirà⟶ dopodiche possiamo soddisfare il credito resuiduo di ciascuno dei rimanenti

---

la sequenza 3,2,1,4,5 consente il soddisfacimento di tutte le richieste (livello grafico di ciò detto sopra)

Capitale Iniziale	IC	150

Saldo di cassa	SC	0

la sequenza 3,2,1,4,5 consente il soddisfacimento di tutte le richieste

Cliente	MAX	Prestito Attuale	Credito residuo	
i	c_i	p_i	n_i	avail[i]
3	30	30	0	0
2	20	10	10	30
1	100	70	30	40
4	50	10	40	110
5	70	30	40	120

Algoritmo del banchiere - Esempio Stato UNSAFE

Capitale Iniziale	IC	150

Saldo di cassa	SC	10

Cliente	MAX	Prestito Attuale	Credito residuo
i	c_i	p_i	n_i
1	100	65	35
2	20	10	10
3	30	5	25
4	50	15	35
5	70	35	35

OSSERVO COME PRIMA COSA IL SALDO CASSA⟶ dopo osservo il processo con prestito attuale più basso

Capitale Iniziale	IC	150

Saldo di cassa	SC	10

se esistesse una sequenza, questa sarebbe 2,3,1,5,4

in evidenza e corsivo sono indicati i casi nei quali la condizione di safety fallisce

Cliente	MAX	Prestito Attuale	Credito residuo	avail[i]
i	c_i	p_i	n_i	
2	20	10	10	10
3	30	5	*25*	*20*
1	100	65	*35*	*25*
5	70	35	35	100
4	50	15	35	135

osservo che il processo 2 riesce a restituire il suo saldo di 20 , portando SALDO CASSA a 20, che però risulta INSUFFICIENTE per soddisfare le richieste dei processi seguenti

Capitale Iniziale	IC	150

Saldo di cassa	SC	0

UNSAFE non implica deadlock

se il cliente 5 restituisce il suo prestito di 35 euro la situazione ritorna SAFE

Cliente	MAX	Prestito Attuale	Credito residuo	avail[i]
i	c_i	p_i	n_i	
2	20	10	10	45
3	30	5	25	55
1	100	75	35	60
4	50	15	35	135
5	70	0	70	150

La similitudine fra banchieri e sistemi operativi ora è chiara...
il denaro sono le risorse ⟶ il sistema le deve allocare ai processi senza che si possa verificare deadlock
le definizioni viste fino a questo punto riguardano il caso teorico elementare di un sistema avente un'unica classe di risorse

Algoritmo del banchiere multi-valuta

è l'estensione del problema del banchiere ⇒ si ipotizza che il banchiere debba fare

prestiti usando valute diverse (euro, dollari, yen, etc.) le diverse valute rappresentano diverse classi di risorse

N:	numero dei clienti
IC_k:	capitale iniziale della valuta k
$c_{i,k}$: $(c_{i,k} < IC_k)$	limite di credito in valuta k del cliente i
$p_{i,k}$:	denaro prestato in valuta k al cliente i $(p_{i,k} \leq c_{i,k})$
$n_{i,k} = c_{i,k} - p_{i,k}$	credito residuo in valuta k del cliente i
$SC_k = IC - \sum_{i=1..N} p_{i,k}$	saldo di cassa in valuta k

Definizione: Stato SAFE

sia *s* una permutazione dei valori *1...N*

esempio, con *N*=*4*: *s = <1 , 3 , 4 , 2>*

⇒ indichiamo con *s(i)* l'i-esima posizione della sequenza

si calcoli il vettore *avail(k)* come segue

- **avail(k)[1]** = **SC(k)**
- **avail(k)[j+1]** = **avail(k)[j]** + p s(j) , con j=1...N-1

uno stato del sistema si dice safe se vale la seguente condizione:

n s(j),k ≤ avail k [j], con j=1...N

Problema

LA REGOLA DI ORDINARE I PROCESSI SECONDO I VALORI DI n(i) NON E' APPLICABILE

l'ordine può essere in generale diverso fra le diverse valute gestite dal banchiere

Soluzione

si può creare la sequenza procedendo passo passo aggiungendo un processo a caso fra quelli completamente soddisfacibili

ovvero, al passo j si sceglie quelli per cui

n s(j),k ≤ avail k [j]

Prestiti Risorse $p_{i,k}$

Cliente	A	B	C
C1	0	1	0
C2	2	0	0
C3	3	0	2
C4	2	1	1
C5	0	0	2

Massimo Richieste $c_{i,k}$

Cliente	A	B	C
C1	7	5	3
C2	3	2	2
C3	9	0	2
C4	2	2	2
C5	4	3	3

Saldo in Cassa $SC_{i,k}$

A	B	C
3	3	2